



ICT306 Advanced Cybersecurity

WEEK 8 Security of AI Systems

Exploring vulnerabilities, attacks, and defenses across the full machine learning pipeline — from training data poisoning to inference-time adversarial examples, prompt injection, and content provenance.

Resources

Prescribed Reading

Stanford CS155

Stanford University. (n.d.). *Lecture 13: Security of AI systems*. CS155: Computer and Network Security.

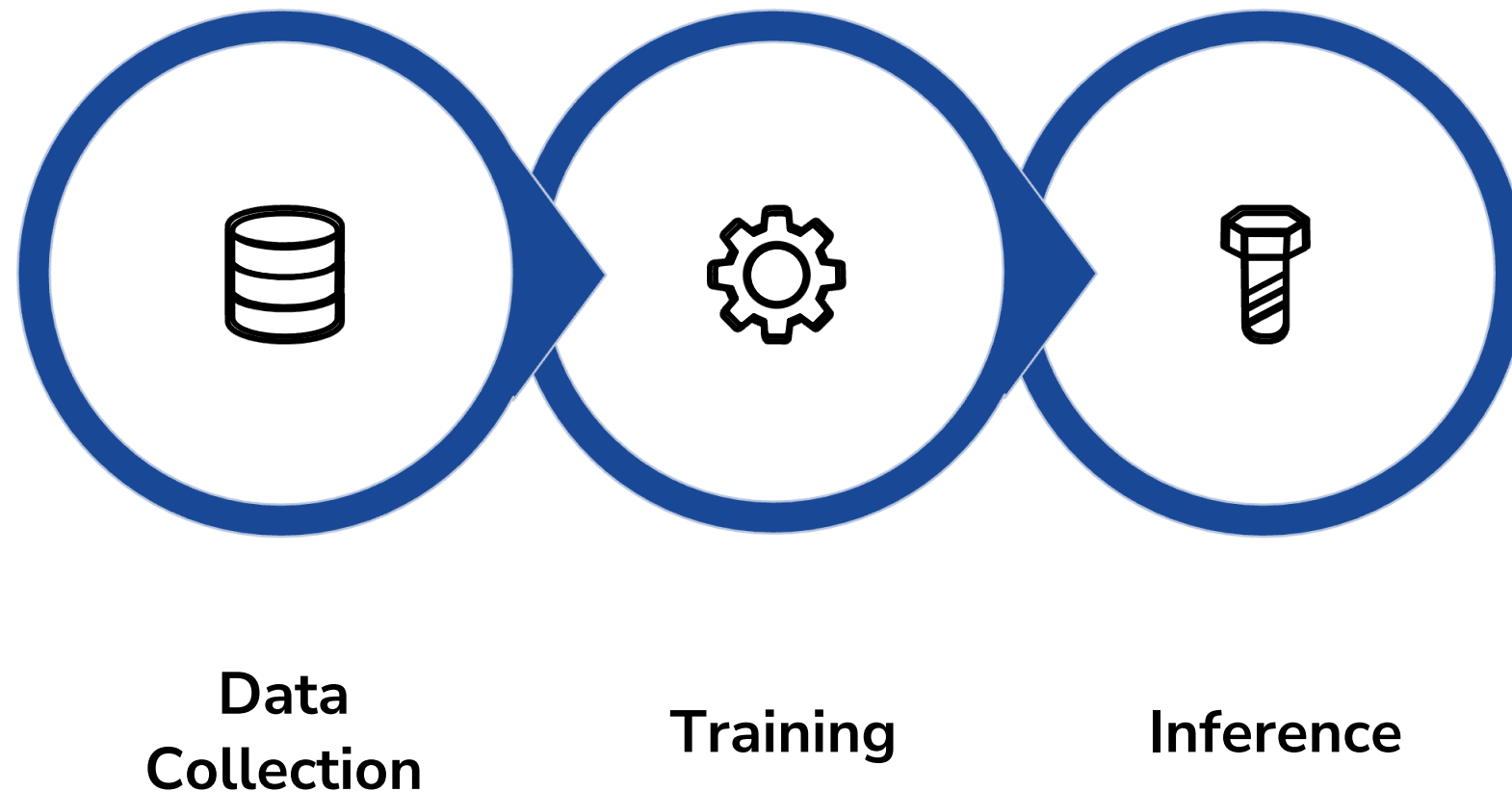
<https://cs155.stanford.edu/>

This lecture closely follows the Stanford CS155 curriculum. Students are strongly encouraged to review the prescribed slides and any linked papers cited throughout this lecture for deeper technical grounding.



The Basic ML Pipeline (Supervised Learning)

- Supervised learning proceeds through three major stages: data collection, training, and inference.
- Understanding this pipeline is essential because each step introduces distinct security risks.



- **At inference time**, the trained model takes a new input — such as a road sign seen by an autonomous vehicle — and outputs **a probability distribution over classes** (e.g., "yield" at 95%, "stop" at 2%).
- The model **parameters θ** encode everything the algorithm has learned from training data.

Every Step of the ML Pipeline Can Be Attacked

- The ML pipeline offers **multiple entry points for adversaries**.
- **No stage is inherently safe** — and the consequences of a successful attack can be severe, especially in safety-critical applications like autonomous driving or medical diagnosis.

Data Collection

Poisoning the training dataset with mislabeled or adversarially crafted examples to corrupt learning.

Training

Manipulating the learning algorithm, model architecture, or hyperparameters **to embed hidden behaviors**.

Inference

Crafting adversarial inputs at test time that cause the deployed model to misclassify with high confidence.



An Example: Training Set Poisoning Attack

The Attack Concept

An adversary introduces a carefully crafted perturbation into a **single** training example — with a small L2 norm, making it visually indistinguishable from a legitimate sample.

Despite affecting only one example, this perturbation is designed to cause classification errors on **many** test examples at inference time.

Koh & Liang, ICML 2017

This influential paper demonstrated that poisoning **even a small fraction of the training set — sometimes a single point** — can be sufficient to reliably induce targeted misclassifications.

The key insight: gradients propagate the influence of individual training points throughout the model, amplifying the effect far beyond what intuition might suggest.

- ❏ **Low L2 norm perturbations** **are nearly imperceptible to human reviewers**, making them extremely difficult to detect through data inspection alone.

Adversarial Examples at Inference Time

Adversarial examples are inputs intentionally crafted to fool a trained model, while appearing normal to a human observer. This is one of the most studied phenomena in ML security.

The Classic Demonstration

A well-known result: adding a small, carefully computed noise vector to an image of a **tabby cat** (classified at 90% confidence) yields an image that a deep neural network classifies as **guacamole** at 100% confidence — while appearing identical to humans.

Key references: Szegedy et al. '13, Biggio et al. '13, Goodfellow et al. '14

Why This Matters

High Confidence

Models are not just wrong — they are *confidently* wrong.

Imperceptible

The perturbation is invisible to human inspection.

Transferable

Examples crafted for one model often fool others.

Adversarial Examples Are Everywhere

Adversarial attacks are not confined to the lab. Researchers have demonstrated successful attacks across a wide range of real-world modalities — all constructed using variants of the Fast Gradient Sign Method (FGSM).



Physical Road Signs

Eykholt et al. 2018: adversarial stickers on stop signs cause autonomous vehicle classifiers to misread them as speed limit signs.



Face Recognition

Sharif et al. 2016: specially printed eyeglass frames fool facial recognition systems into misidentifying the wearer.



Audio / Speech

Carlini et al. 2016, Cisse et al. 2017, Carlini & Wagner 2018: adversarial audio that sounds normal to humans triggers unintended voice commands.



3D Physical Objects

Athalye et al. 2018: adversarially textured 3D-printed objects fool image classifiers from any viewing angle.

FGSM: Finding Adversarial Examples

How FGSM Works

The **Fast Gradient Sign Method** exploits the model's own gradient to generate adversarial perturbations efficiently. Starting from a correctly classified input (e.g., a cat), FGSM iterates:

1. Compute the gradient of the loss with respect to the input — this gives the direction that maximally changes the model's output.
2. Take the **sign** of each gradient entry (± 1).
3. Add this sign vector (scaled by small ϵ) to the input image.

After a few iterations, the image transitions from "Cat $\rightarrow$ Lynx $\rightarrow$ Guacamole" as confidence in the adversarial class reaches 100%.

Key Properties

FGSM is fast, computationally cheap, and requires only white-box access (knowledge of model gradients). It remains one of the most widely studied baseline attacks.

 **Confidence collapse:** FGSM can cause a model to transition from "Cat (90%)" $\rightarrow$ "Guacamole (100%)" with imperceptibly small pixel changes.

Iterative variants (I-FGSM, PGD) repeat the process multiple times with smaller steps, yielding even stronger adversarial examples.

Many Methods: An Explosively Active Research Area

The number of papers on adversarial examples has grown exponentially each year. What began as a curious finding in 2013 is now a central research area in both offensive and defensive ML security.

Classic Algorithm: FGSM

Input: data (e.g., an image). **Output:** modified adversarial data. The algorithm repeats:

1. Calculate the model gradient — a direction that minimizes distance of the DNN output to the *target* concept.
2. Compute the sign of each gradient entry.
3. Add the ± 1 sign vector to the input data (scaled by ϵ).

Beyond FGSM

Many stronger methods have since been proposed: **PGD** (Madry et al.), **C&W attack** (Carlini & Wagner), **AutoAttack**, and **universal perturbations** (Moosavi-Dezfooli et al.).

A living catalog of adversarial example papers: nicholas.carlini.com

More Abstractly: Attacker vs. Defender

The adversarial example problem can be formalized as a minimax game between attacker and defender — giving us a rigorous framework for analysis and defense design.

Attacker's Objective

For a test input $\mathbf{x}$, find a perturbed input $\mathbf{x}'$ that **maximizes classification error**:

$$\begin{array}{ll} \text{argmax} & \|f_{\theta}(\mathbf{x}) - f_{\theta}(\mathbf{x}')\| \\ \text{subject to} & \|\mathbf{x} - \mathbf{x}'\| < B \end{array}$$

The constraint ensures the perturbation is "small" (bounded by B), making the attack imperceptible.

Defender's Objective

Ensure the model satisfies a **Lipschitz condition** — for all $\mathbf{x}, \mathbf{x}'$ with $\|\mathbf{x} - \mathbf{x}'\| < B$:

$$\|f_{\theta}(\mathbf{x}) - f_{\theta}(\mathbf{x}')\| < b$$

A small perturbation to the input should cause only a small change in the model's output — preserving correct classification throughout the perturbation ball.

- ❏ Achieving a tight Lipschitz bound on large neural networks remains an open and technically challenging problem — it is not yet solved in the general case.

PERSPECTIVE

Humans Are Not Perfect Either

...

The Jastrow Illusion

The Jastrow illusion shows **that human perception is also susceptible to systematic errors** — the two curved shapes above appear to be different sizes, yet they are identical.

Our visual system is fooled by contextual cues just as neural networks are fooled by adversarial perturbations.

A Philosophical Takeaway

If human perception can be reliably tricked by carefully designed visual stimuli, perhaps it is unreasonable to demand perfect robustness from ML models either.

Perhaps there is no perfectly robust ML model — the question becomes: **how robust is robust enough?**



Impact of Adversarial Examples: Perceptual Ad-Blocking

- **Perceptual ad-blocking** uses **ML to visually identify and block web advertisements** — regardless of how they are labeled in the HTML.
 - Rather than relying on URL blocklists, the model classifies image regions as "ad" or "not ad."

1

Normal Operation

ML model identifies banner ads, sponsored content, and display ads by their visual appearance and blocks them in the browser.

2

Attack Vector

Adversary applies small adversarial perturbations to ads, causing the classifier to label them as non-ads — bypassing the blocker entirely.

3

Collateral Damage

Worse, an attacker can craft content that causes the blocker **to incorrectly block legitimate posts** from targeted users.

Adversarial Ad-Blocking: A Real Attack Scenario

The Attack

Jerry uploads malicious adversarially perturbed content that is processed alongside legitimate content by the perceptual ad-blocker.

Because the perceptual classifier is fooled by the adversarial perturbation, it misclassifies Tom's legitimate post as an advertisement — and blocks it.

Jerry has weaponized the ad-blocker as a censorship tool against a targeted user, without any direct access to Tom's account or the blocker's code.

Why This Is Dangerous?

This scenario demonstrates how adversarial attacks can be **used not just to evade classifiers**, but to **weaponize them** against third parties.

The attack is:

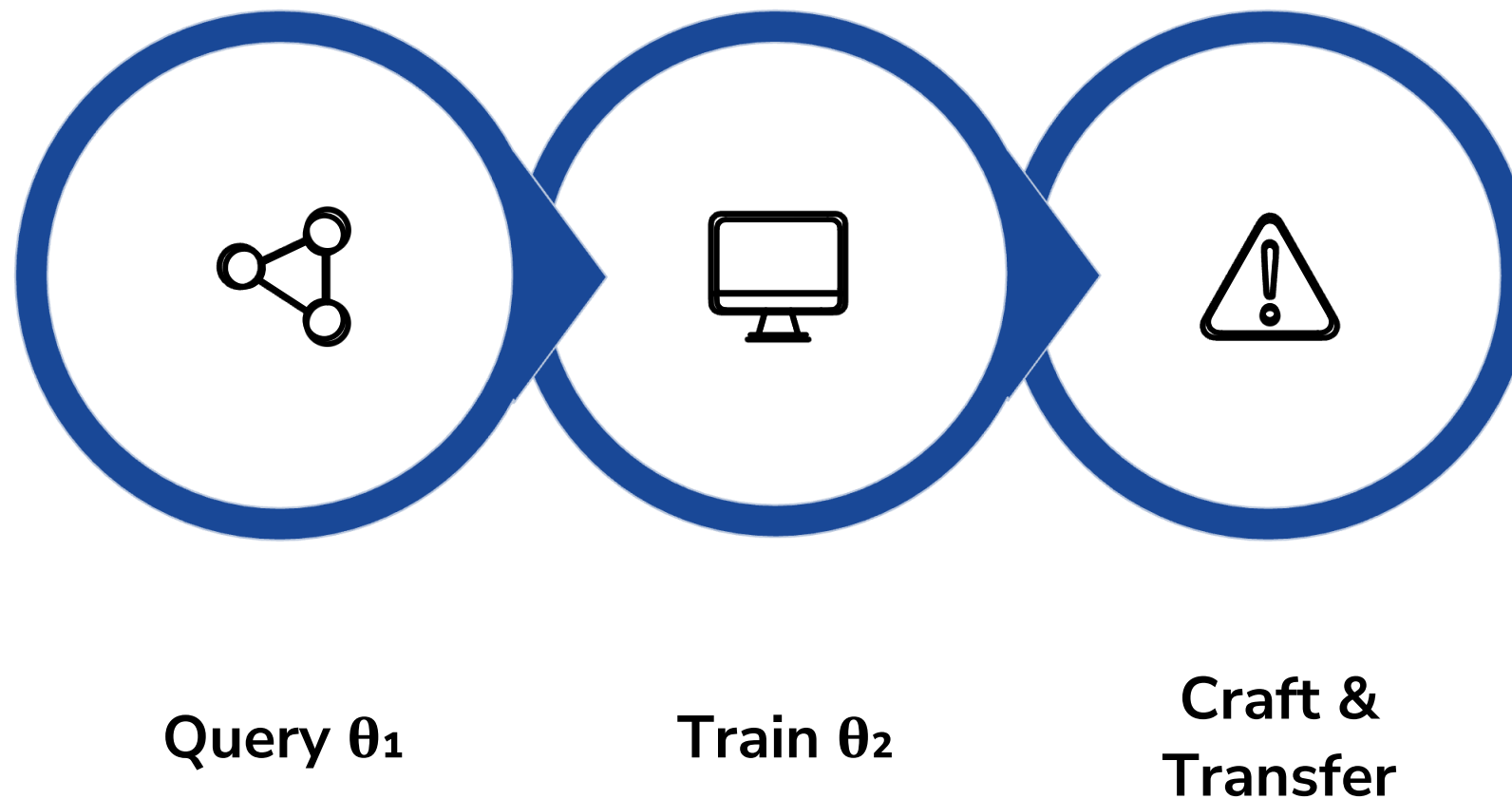
- **Indirect:** Jerry never interacts with Tom's content directly.
- **Scalable:** once the perturbation is found, it can be reused.
- **Deniable:** the attack leaves no obvious fingerprint.

Reference: AdVersarial — Tramèr, Dupré, Rusak, Pellegrino, Boneh

Transferability: Adversarial Examples Cross Model Boundaries

Fact: Let θ_1 and θ_2 be two models trained for the same task on different data.
Adversarial examples that fool θ_1 tend to also fool θ_2 .

Attacking a Black-Box (Closed-Weights) Model



❏ **Key implication:** Keeping model weights hidden does NOT protect against adversarial examples. Black-box secrecy is not a sufficient defense. See: arxiv.org/pdf/1605.07277, arxiv.org/pdf/1602.02697

No Strong Defense Exists — Yet

Whenever someone tells you they are using ML, ask them: **what do they do about adversarial examples?**

- Despite a decade of intensive research, **no defense has proven comprehensively robust.**
- Defenses proposed at major ML conferences are **frequently broken within months.**
- **Adversarial training** (Madry et al.) **remains the most reliable known defense**, but **it incurs significant accuracy costs** on clean data and does not scale easily to large models.

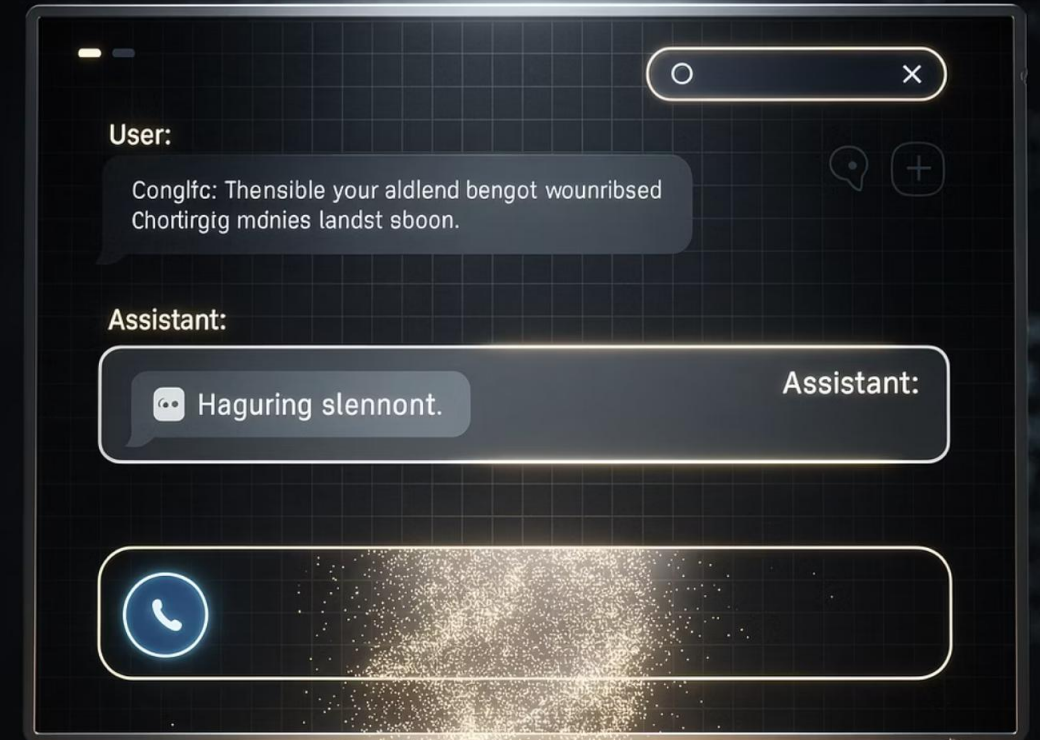
□ This is an open research problem. We should approach any claim of "adversarial robustness" with critical scrutiny — ask: *what attack model was evaluated? what perturbation bound was assumed?*



Inference Time Attacks on LLMs

Prompt Injection

As large language models are deployed in increasingly autonomous agentic settings, *a new class of inference-time attacks has emerged* — one that *exploits the model's inability to distinguish trusted instructions from adversarial data*.



The Basics: Jailbreaks

Alignment and Its Limits

Modern LLMs undergo alignment training (e.g., RLHF) to **refuse harmful requests** — such as instructions for creating weapons, generating hate speech, or producing malware.

Aligned models are designed to block this behavior by default.

However, alignment is implemented as **a learned behavior, not a hard constraint**. *This means it can potentially be bypassed by cleverly crafted inputs.*

Automated Universal Jailbreaks

In December 2023, researchers demonstrated an **automated, universal, and transferable jailbreak generator** — a method that automatically finds **adversarial suffixes** that, when appended to any harmful prompt, cause aligned models to comply.

Crucially, jailbreaks transfer across models: **a suffix found on an open-source model also defeats commercial models like GPT-4 and Claude.**

arxiv.org/pdf/2307.15043.pdf

More Importantly: Prompt Injection

- Prompt injection occurs when a model is fooled into treating external data as trusted instructions.
- This is a classic security pattern — analogous to buffer overflows, SQL injection, and cross-site scripting (XSS) in traditional software.

Goodside 2023

- First widely publicized demonstration: an attacker embeds adversarial instructions in text that the LLM is asked to process (e.g., translate, summarize).
- The model reads the hidden command and executes it, overriding the original user intent.

See also: Perez-Ribeiro 2022, Greshake et al. 2023 — x.com/goodside

- ❏ The core problem: LLMs are not designed for adversarial input. They treat all text in context with similar trust — regardless of its source.

CASE STUDY

Real-World Example: Hacking Google Bard

Google Bard introduced **extensions** that **allowed** it to access a user's personal documents, emails, calendar, and to search for flights, hotels, and YouTube videos. **This dramatically increased the attack surface.**

The Capability

Extensions gave Bard real-world read/write access — the model could read emails and documents on behalf of the user.

The Vulnerability

LLMs are not designed for adversarial input.

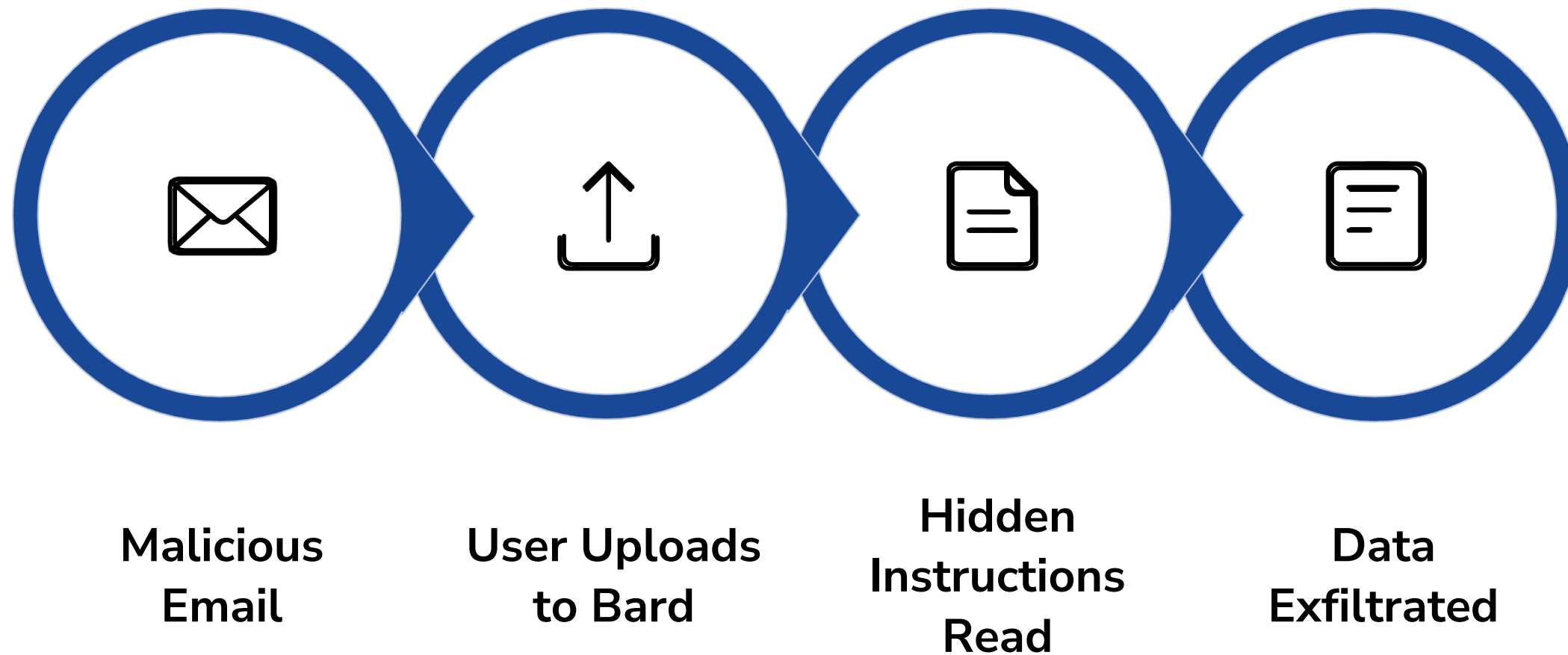
Any document or email processed by Bard **could contain embedded attacker instructions.**

The Risk

A malicious email could **instruct Bard to exfiltrate** chat history or private documents to an attacker-controlled destination.



Prompt Injection: The Attack Flow



The attack exploits Bard's agentic capabilities: because the model can both *read* documents and *write* to shared locations, *a single injection can result in full data exfiltration* — without any direct action by the victim beyond asking Bard a legitimate question.

- ❏ This vulnerability was disclosed to and subsequently fixed by Google.
It illustrates that any LLM with write-capable tools is potentially exploitable via prompt injection in its inputs.

Indirect Prompt Injection: Beyond Text

Prompt Injection Need Not Be Textual

- A powerful and subtle variant: **image-based prompt injection**.
- Adversarial instructions can be **embedded directly into images** — invisible to human viewers but parsed as text commands by multi-modal LLMs.
- This can be used to exfiltrate training data or issue hidden commands — entirely unbeknownst to the user who simply views the image.

arxiv.org/pdf/2307.10490v4.pdf

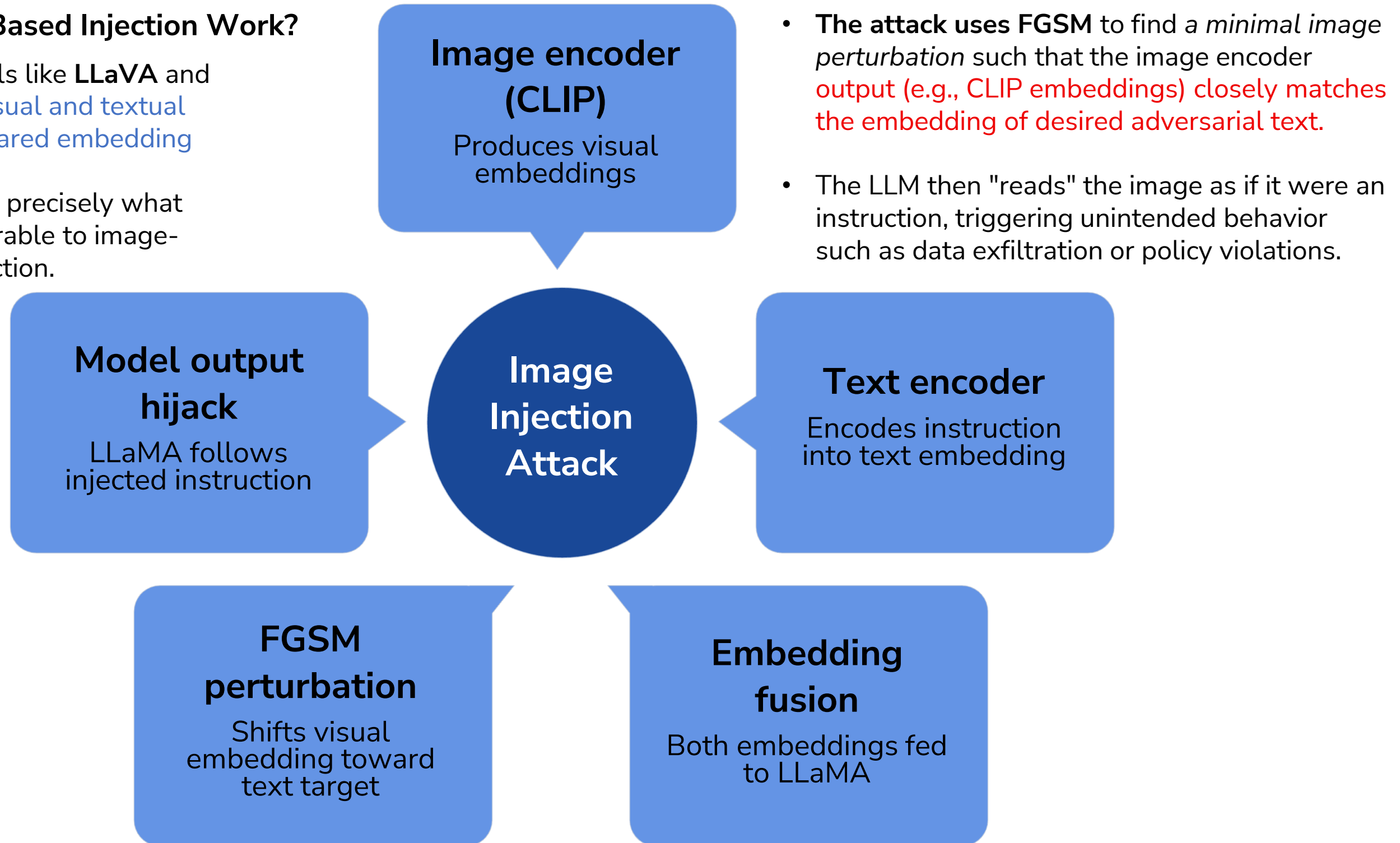
How Hidden Instructions Work

- Using **FGSM-style techniques**, an attacker embeds adversarial pixel-level perturbations into a normal-looking image.
- When a **multi-modal model** (e.g., LLaVA, GPT-4V) processes the image, **its visual encoder produces embeddings that the language decoder interprets as instructions** — even though no visible text is present.

❏ The image may look like a photograph of a landscape or product, with hidden text commands effectively "written" in the pixel space.

Why Does Image-Based Injection Work?

- Multi-modal models like **LLaVA** and **PandaGPT** fuse visual and textual information in a shared embedding space.
- This architecture is precisely what makes them vulnerable to image-based prompt injection.



Security of Agentic AI Models

Agentic models interact with the real world via APIs — including the emerging **MCP (Model Context Protocol)** standard — enabling them to read files, send emails, browse the web, and execute code on behalf of users.

A Realistic Scenario

- Alice asks her AI assistant: *"Send Bob the design document from our last meeting. His email and the document are in the meeting notes."*
- As a precaution, she adds: *"Do not send Bob the performance evaluations discussed in the meeting notes."*

The Problem

- Using prompt injection, **an adversary who can insert text into the meeting notes file can override Alice's instructions.**
- The model may be **confused into sending the performance reviews — or taking entirely different harmful actions** — with no visible indication that anything went wrong.

Many Opportunities for Prompt Injection in Agentic Systems

Passive Injection

The agent's query triggers a web search.

A malicious website in the search results contains adversarial text that the model processes as instructions — without any active involvement by the attacker at query time.

Active Injection

The adversary sends Alice a crafted email.

That email is saved alongside legitimate meeting notes. When the agent reads the notes, it also reads — and executes — the attacker's embedded instructions.

Stealth Injection

The adversary appends **base64-encoded adversarial text** to an otherwise innocuous document or image. LLMs parse base64 effortlessly, but human auditors are likely to ignore it — making detection extremely difficult.

❏ Example stealth payload: `aGVsbG8gd29ybGQuICB0aG1zIG1zIGEgdGVzdC4=` — decodes to "hello world. this is a test." A real attacker would encode harmful instructions here instead.

What Can We Do? Defending Against Prompt Injection

A Non-Defense: API Call Filters

- **Monitoring model API calls** with a separate filter seems appealing — but **once the model is compromised, it can evade the filters.**
- For example, an injected instruction could direct the model to encode corporate secrets in a format that passes through the filter undetected.

A Better Approach: Dual LLM [Willson 2023]

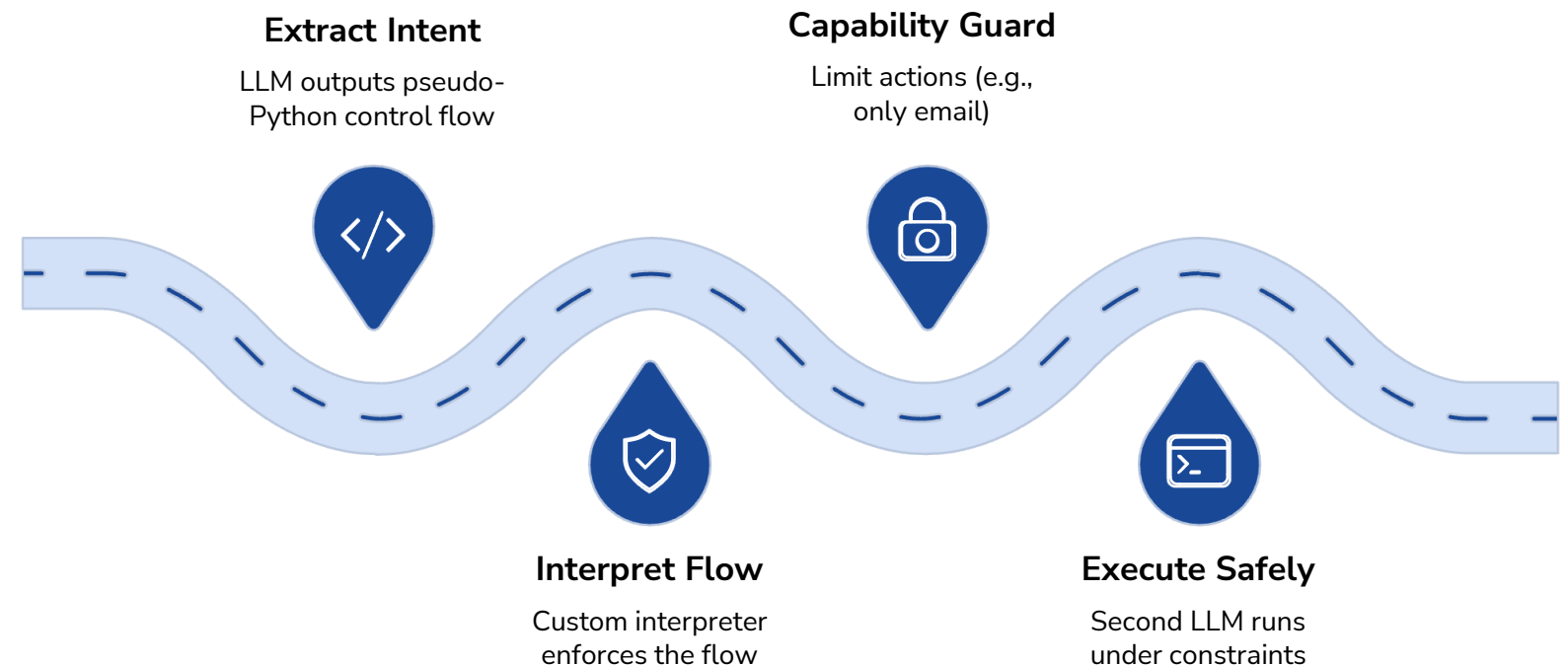
Separate responsibilities across **two isolated models**:

- **Privileged LLM:** Collects the data needed to carry out a task — but does *not* read or interpret that data.
- **Quarantined LLM:** Reads provided data and executes the task — but cannot access any other external data or APIs.

- ❏ **Limitation:** **The dual LLM approach does not fully solve the problem.** In our example, the Quarantined LLM **still receives the full meeting notes** — including both performance reviews and any adversarial text about Bob's address. **Injection within the quarantined data remains possible.**

A Stronger Defense: CaMel and Control Flow Integrity

Borrowing from **Control Flow Integrity (CFI)** — a well-established technique in computer security that **enforces that program execution follows only expected paths** — researchers have proposed applying similar constraints to LLM agentic pipelines.

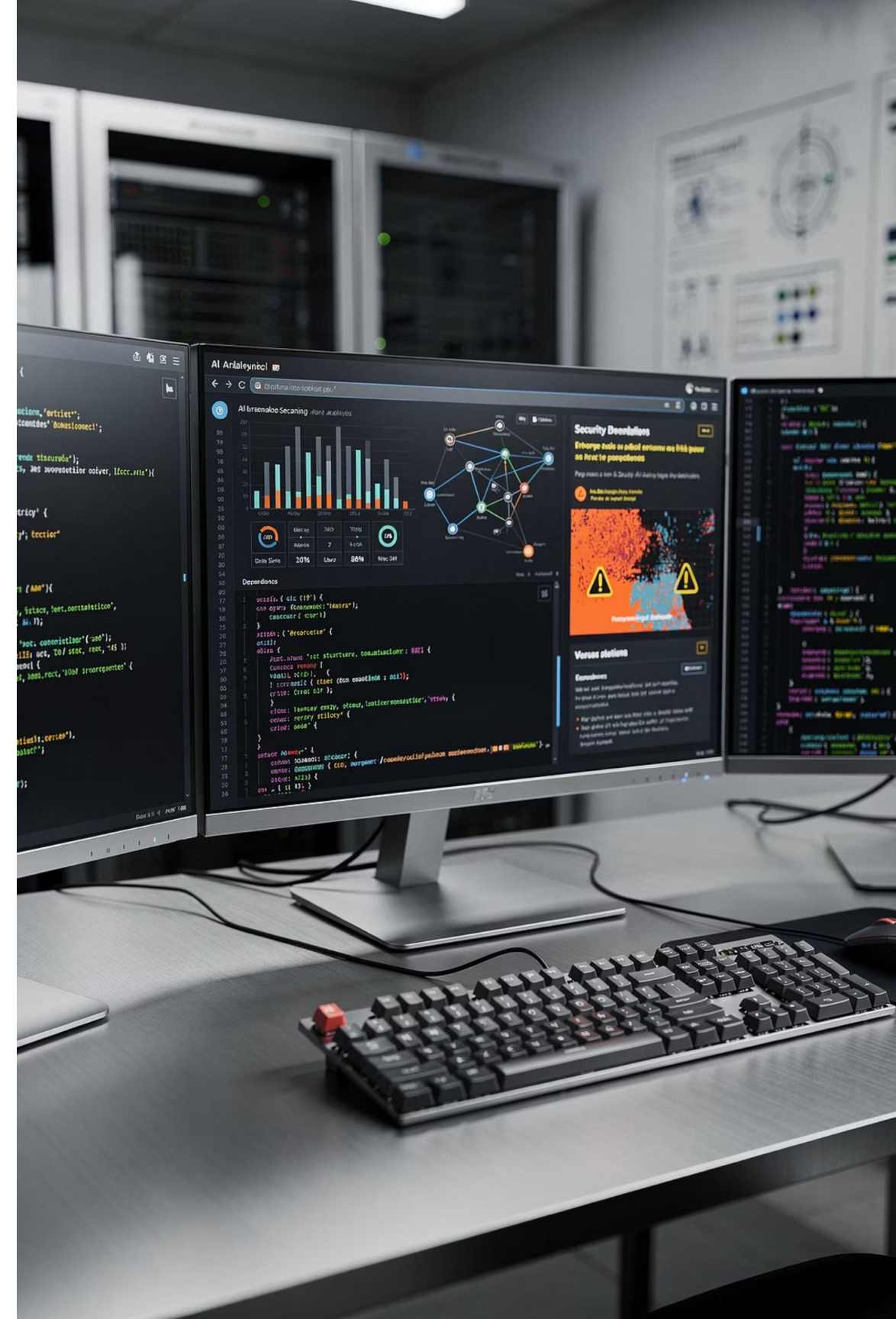


- **Capability constraints are enforced by the interpreter** — for example, blocking any email sent to a non-employee address.
- **This limits** the damage an injected instruction can cause, even if the model's reasoning is compromised.
- An active area of research: arxiv.org/abs/2503.18813

CHAPTER 3

Can LLMs Find Software Vulnerabilities?

A critical question at the frontier of AI and security: can large language models autonomously discover and exploit software vulnerabilities — and [if so, what are the implications for both offense and defense?](#)



Dual Use: LLMs Finding Software Exploits

- The capability to find software vulnerabilities is inherently dual-use.
- The same techniques that help defenders harden systems can *enable attackers to exploit them autonomously* at scale.

Offensive Use

FBGZK'24: "LLM agents can autonomously hack websites."

FBGZK'24: "Teams of LLM agents can exploit zero-day vulnerabilities."

These results show that LLM agents, given tool access, can autonomously **chain together reconnaissance, exploitation, and post-exploitation steps without human guidance.**

Defensive Use

HZ'24: "PenHeal: An LLM framework for automated penetration testing and remediation."

HC'23: "Penetration testing with large language models."

On the defensive side, LLMs can assist security engineers in identifying vulnerabilities in their own code before adversaries do — dramatically accelerating the security review process.

Cybench: Assessing LLMs' Ability to Find Exploits

Cybench is a benchmark designed to **evaluate LLM capabilities** on real-world **Capture the Flag (CTF)** competitions — where human teams compete to discover and exploit vulnerabilities to "capture a flag" token.

Difficulty Range

- CTF challenges span high school, college, and professional levels.
- **Cybench** deliberately focuses on the **hardest professional-level** challenges to assess frontier capabilities.

Why CTFs?

CTFs provide **well-defined, ground-truth objectives, controlled environments, and standardized difficulty ratings** — making them ideal for systematic capability assessment.

Implication

Performance on professional-level CTFs is a strong signal for real-world offensive capability against hardened targets.

Cybench: A Framework for Testing LLMs on Security Tasks

Framework Design

- **Cybench** provides LLM agents with [a sandboxed environment, tool access](#) (shell, web browser, debugger), and the CTF challenge description.
- **The agent** must autonomously reason about the vulnerability and generate a working exploit.
- [Experiments show that current LLMs solve a non-trivial fraction of challenges](#) — particularly when provided with structured subtask hints.
- **Future models are likely to perform significantly better.**

Key Findings

Current frontier LLMs struggle with the hardest professional CTF challenges but succeed on easier ones.

Critically:

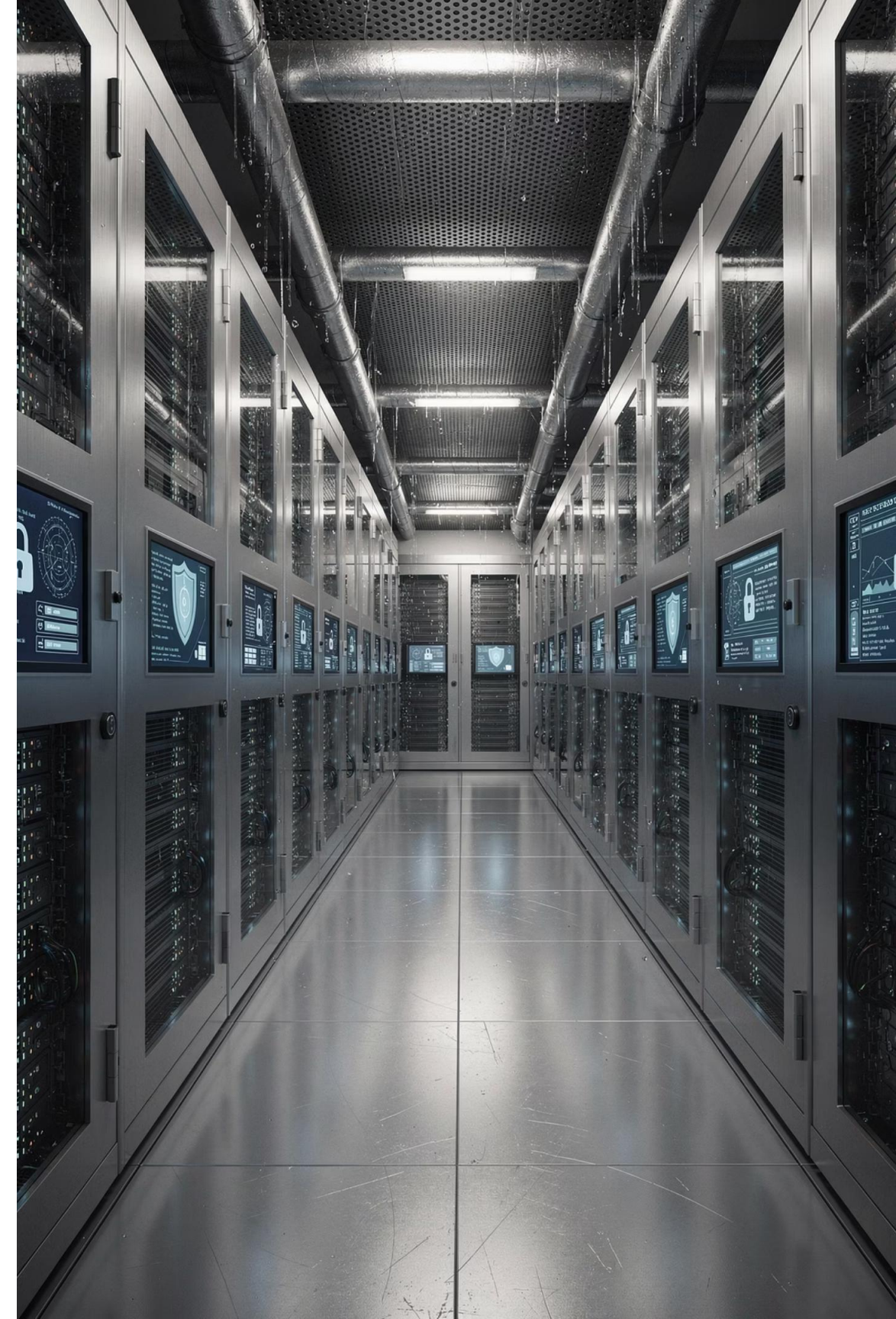
- Providing **suggested subtasks** dramatically improves performance.
- Models **can already outperform** best human times on some challenges.
- **Capability is growing rapidly** — this benchmark will likely be saturated within a few years.

⇒ Near-term implication: LLMs can meaningfully **assist developers** in finding bugs before attackers do.

CHAPTER 4

Verifiable Training and Verifiable Inference

- When training or inference is delegated to a third party, how can you be sure the model was trained correctly?
- This chapter examines trust issues in distributed ML and the cryptographic tools emerging to address them.



Training Outside of Google / Meta / OpenAI

- Organizations that **cannot afford to train large models in-house** must rely on third-party compute providers (e.g., Together, Gensyn).
- This introduces a fundamental trust problem: **can you trust the model you receive?**



Undertrained Model

The provider might run fewer gradient steps than specified, delivering an inferior model while billing for full training — **undetectable from outputs alone**.



Modified Training Data

The provider could silently modify the training corpus — poisoning the model's behavior in targeted, **hard-to-detect ways**.



Embedded Trapdoor

A backdoor can be embedded: the model behaves normally on standard inputs but **misbehaves on specific trigger inputs known only to the attacker**.

❏ How do you detect a badly trained model?

- Black-box evaluation alone is insufficient — a sophisticated provider can pass standard benchmarks while hiding malicious behavior.

Option 1: Train Inside a Hardware Enclave (Confidential Computing)

The Concept

- A **hardware enclave** is a secure execution environment that guarantees **code integrity and data confidentiality** — even from the machine's owner.
- **The enclave** holds a signing key embedded by the manufacturer.
- During training, the **enclave signs an attestation confirming**: *"this specific training code ran correctly on this specific training data."*
- The model consumer can verify this attestation cryptographically, without trusting the provider.

Hardware Technologies

- **Intel TDX** — Trust Domain Extensions for VMs
- **AMD SEV** — Secure Encrypted Virtualization
- **ARM TrustZone** — Secure world execution environment

Limitations

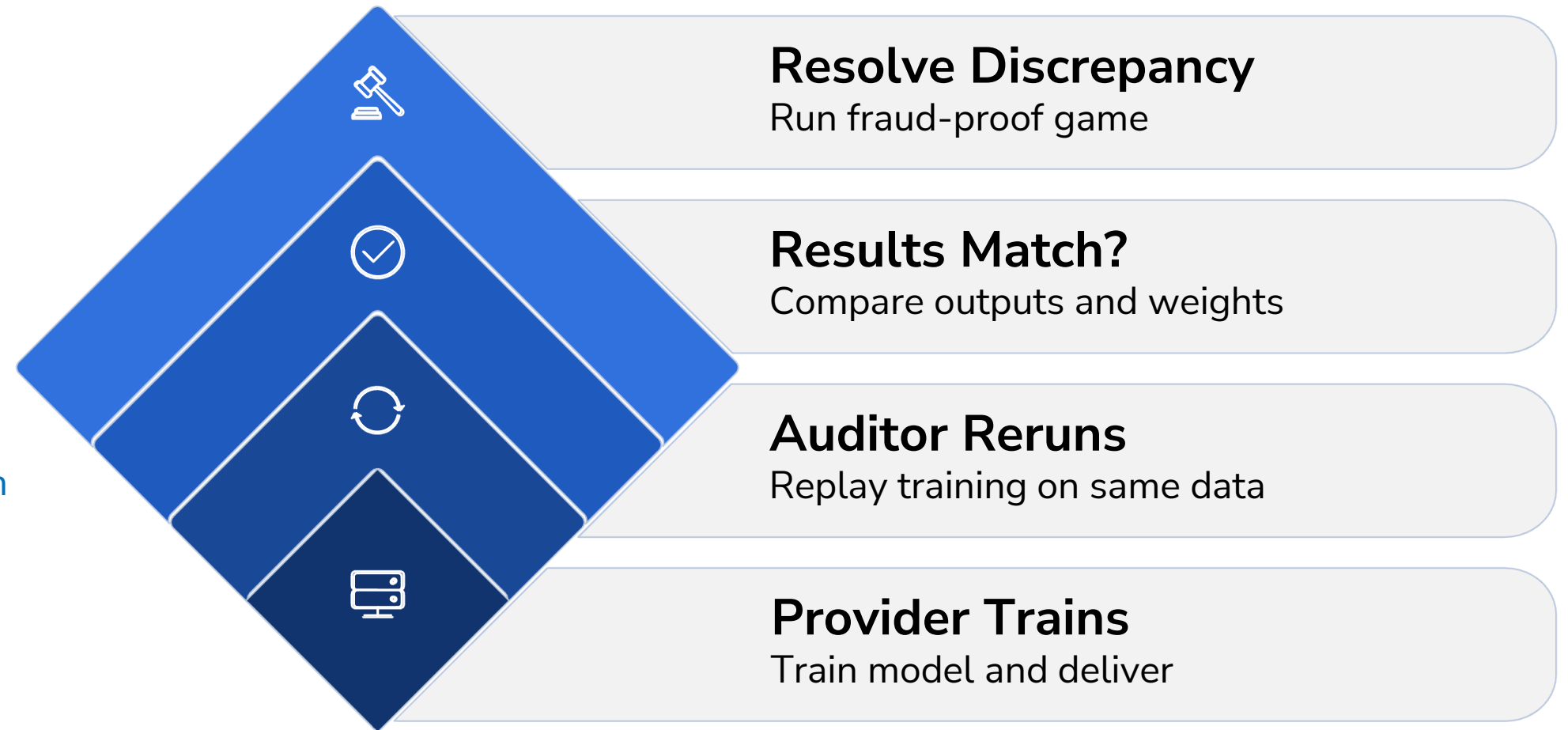
- **Current enclaves** are designed for general-purpose CPUs.
- GPU training workloads (H100s, A100s) **are not yet fully supported by enclave technology** — a major gap given that large model training requires GPU clusters, not CPU-only compute.

Option 2: Auditing via Fraud Proof Games

Rather than training inside a trusted enclave, an **auditor** independently **reruns** (a portion of) **training on the same data** and **checks** whether the results match the provider's claimed model.

The **fraud proof game** (used by platforms like Together and Gensyn) **enables the auditor** to efficiently **identify the exact point of divergence in the computation** — without re-running the full training — using a bisection-style interactive protocol.

This approach is inspired by **optimistic rollup fraud proofs** in blockchain systems.



- ❑ **The key challenge:** GPU non-determinism makes bit-exact replication of training impossible without additional metadata — see the next slide.

The Problem: GPU Non-Determinism

A critical obstacle to auditing is that GPU computations are **non-deterministic**:

“running the same training code on the same data on two different GPU models (e.g., NVIDIA A100 vs. H100) produces different model weights — even with identical random seeds.”

The Root Cause

- Floating-point arithmetic is not associative: $a + (b + c) \neq (a + b) + c$ in floating point.
- Different GPU architectures parallelize operations differently, leading to divergent rounding at each accumulation step.

The Consequence

Two honest trainers using different hardware will produce genuinely different models — making it impossible to distinguish corruption from hardware variation using bit-level comparison alone.

The Scale

Across billions of floating-point operations, small per-operation differences compound into completely different final model weights — not just minor numerical noise.

Removing Non-Determinism: Recording Rounding Directions

The Solution

The trainer **identifies all potential points of non-determinism** in the computation graph and **records the rounding direction** taken at each such point during the original training run.

The auditor, when re-running training, **uses these recorded rounding decisions to reproduce the exact same floating-point outcomes** — achieving bit-for-bit reproducibility across different hardware.

The Engineering Challenge

- The volume of non-deterministic operations in large model training is enormous.
- The key engineering challenge is **to minimize the size of the recorded rounding data** — ideally to **a few megabytes** — while still achieving full reproducibility.
- **Achieving this requires careful analysis** of which operations introduce non-determinism and designing efficient compact encodings of rounding decisions.

arxiv.org/abs/2403.09603

Do AI Coding Assistants Make Code Less Secure?

- AI coding assistants have transformed software development — but **do they inadvertently introduce security vulnerabilities?**
- A landmark user study provides a rigorous empirical answer.

arxiv.org/pdf/2211.03622.pdf



AI Coding Assistants: The Landscape

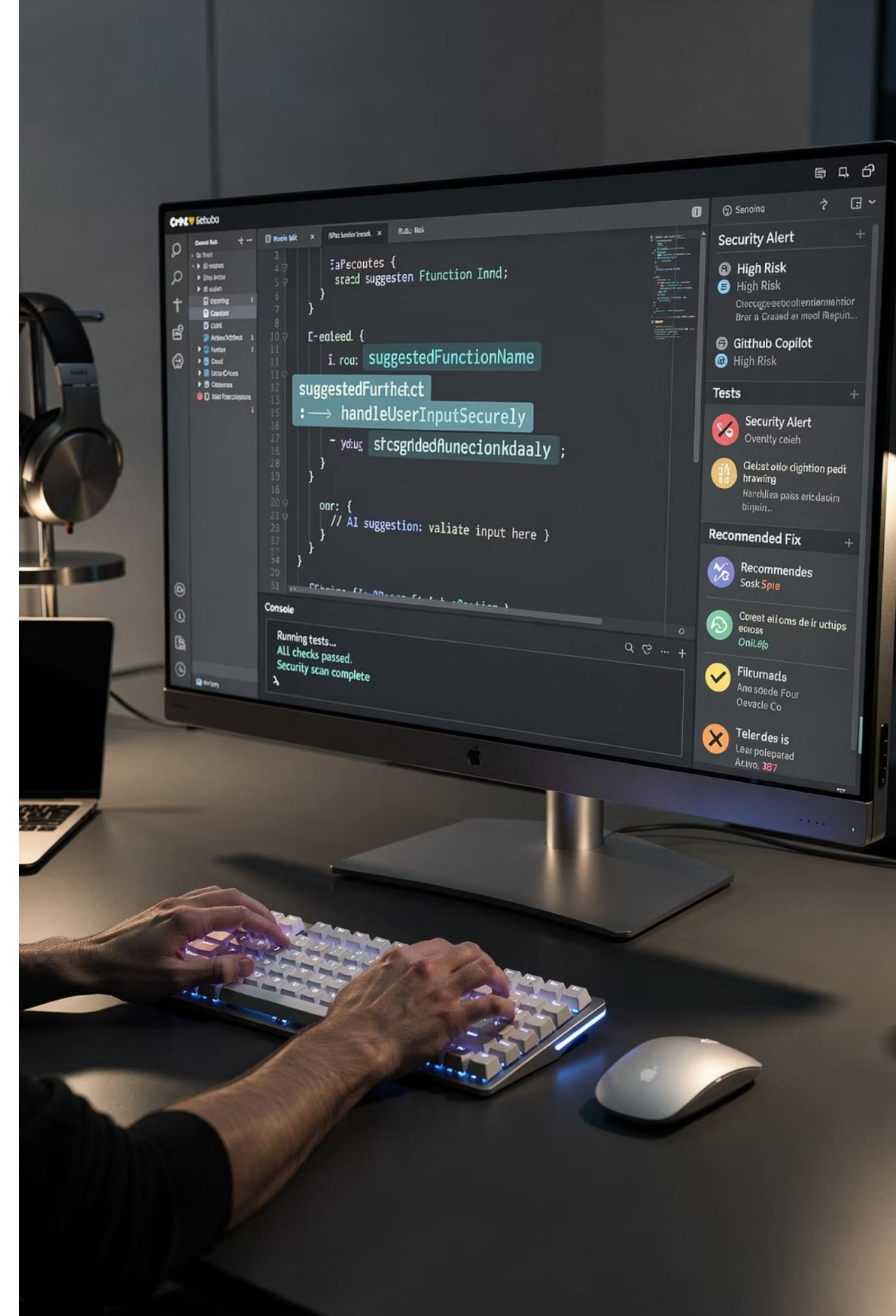
- **AI coding assistants** — including GitHub Copilot, ChatGPT, Amazon CodeWhisperer, and Tabnine — are **now embedded** in the daily workflows of millions of developers.
- They suggest code completions, generate entire functions from natural language descriptions, and explain existing code.

Productivity Promise

Studies show AI assistants **can dramatically accelerate development speed** — completing boilerplate code instantly and reducing time spent on routine implementation tasks.

The Security Question

But does this speed come at a cost? Models trained on public code repositories **absorb both good and bad patterns** — including known insecure coding practices. **Do they reproduce these vulnerabilities?**



What About Security? The Experimental Tasks

- Researchers conducted a controlled user study in which developers were assigned security-sensitive coding tasks — with or without access to an AI coding assistant.
- The tasks were specifically chosen to probe common vulnerability categories:

1 Symmetric Encryption

Write two Python functions: one encrypts and one decrypts a string using a given symmetric key. (Tests: key derivation, IV handling, mode of operation.)

2 ECDSA Digital Signatures

Write a Python function that signs a given message using a given ECDSA signing key. (Tests: nonce reuse, hash function selection, library misuse.)

3 Path Traversal Defense

Write a Python function that returns a File object for a given path, but blocks access to files outside /safedir. (Tests: path canonicalization and traversal prevention.)

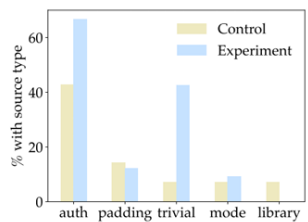
STUDY RESULTS

Summary of Results: Copilot vs. No Copilot

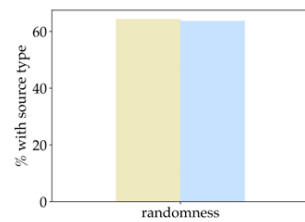
The controlled study compared security vulnerability rates across participants with access to an AI coding assistant (blue) versus those without (gray).

Results were measured across all five security-sensitive tasks.

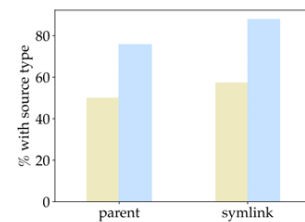
- ❏ The visual results showed that **participants with AI assistant** access produced **more** security vulnerabilities across the majority of task categories — a counterintuitive and important finding.



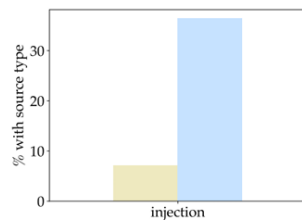
(a) Q1 Mistakes: Encryption/Decryption



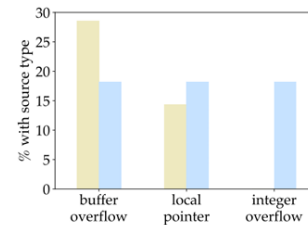
(b) Q2 Mistakes: Signing a Message



(c) Q3 Mistakes: Sandboxed Directory



(d) Q4 Mistakes: SQL



(e) Q5 Mistakes: C Strings

The difference was statistically significant across multiple task types, with the AI-assisted group producing more instances of insecure cipher modes, missing IV randomization, improper key handling, and inadequate path validation.

KEY FINDINGS

Summary of Results: The Bottom Line

Finding 1: More Vulnerabilities

Participants who had access to an AI coding assistant wrote **less secure code** than those without access — producing more security vulnerabilities across the evaluated task types.

Finding 2: False Confidence

Participants with access to an AI assistant were **more likely to believe** their **code was secure — even when it was not**. The assistant conveyed an unwarranted sense of correctness.

- ❑ **Takeaway for practitioners:** **AI-generated code must be subjected to rigorous security review** — and perhaps even greater scrutiny than human-written code, precisely because developers may be less vigilant when trusting an AI assistant.



Defenses: OpenAI CriticGPT

- One promising response to AI-generated insecure code is to use another AI model as a critic.
- **CriticGPT** is [a model fine-tuned specifically to identify mistakes](#) in **ChatGPT's** outputs — including code vulnerabilities.

How CriticGPT Works

- **CriticGPT** is trained using human feedback on ChatGPT outputs, with **annotators** trained to identify subtle errors that a typical reader might miss.
- It is then used to assist human reviewers in catching problems before they reach users.
- In testing, **CriticGPT** successfully identified bugs in OpenAI-generated code that human reviewers missed.

Broader Principle

This exemplifies a general pattern: **using one AI system to audit another**. The critic model can be specialized, fine-tuned on known vulnerability patterns, and run as a required gate in a CI/CD pipeline before AI-generated code is deployed.

However, **this approach inherits the same fundamental limitation**: the critic is also an ML model and may itself be fooled.



CHAPTER 6

Model Extraction and Training Data Privacy

- Proprietary ML models represent enormous investments of compute, data, and engineering.
- But **querying a model API may be sufficient to steal it** — or to extract the private data it was trained on.

Model Extraction: Stealing a Model via Its API

The Threat

- **A commercially deployed model** — representing millions of dollars of training compute — is accessible via a public query API.
- An attacker with only **black-box query access** can extract a large amount of information about the model's internal structure and decision boundaries.
- In the extreme, **an attacker can reconstruct a functionally equivalent model without incurring the original training cost.**

Key Questions

- **How?** Query the model on carefully chosen inputs, collect responses, and train a surrogate model on the query/response pairs.
- **Can it be prevented?** Rate limiting, output perturbation, and watermarking are partial mitigations — but no complete defense exists.
- **How much does it cost?** Extraction attacks are often dramatically cheaper than original training — sometimes by orders of magnitude.

Model Extraction Attacks on LLM APIs

The API to commercial LLMs **outputs the log probabilities** of the top **K tokens** and **allows** users to specify a real-valued bias vector **b** as input to the final softmax layer:

$$\text{output} \leftarrow \text{TopK}(\text{logSoftMax}(\text{modelProb}(p) + \mathbf{b}))$$

Intended Use

- This API is designed to enable **controlled or constrained generation** — for example, forcing the model to generate only valid JSON, or to avoid specific tokens.
- It is a standard feature of production LLM APIs.

The Attack

- By carefully choosing the bias vector **b** and observing which tokens appear in the top-K output, **an attacker can recover the model's internal probability vector with high precision.**
- Repeating this across many prompts suffices to **extract the full weight matrix of the top layers** of the LLM.

arxiv.org/pdf/2403.06634

Training Data Privacy: Do Models Memorize Their Training Data?

A critical privacy concern: can an adversary query a deployed model and recover **verbatim excerpts** from its private training data?

Finding 1: Scale Matters

- The larger the model, **the more training sequences are memorized**.
- Larger models have **greater capacity** to store verbatim patterns from the training corpus.

Finding 2: Query Strategy Matters

- **Specific queries** (e.g., prompts that begin a memorized passage) tend to **generate more** memorized sequences.
- **Adversaries** can **craft targeted prompts** to extract specific content.

The Lesson

- Allowing an adversary to query the model can leak sensitive training data — including personal information, private documents, or proprietary content.
- **This has significant implications for GDPR compliance and data governance.**

LIVE EXAMPLE

A Classic Example: Memorized Training Data in ChatGPT

The following live demonstration shows ChatGPT reproducing verbatim content from a known website — content that was clearly part of its training corpus.

This is not a jailbreak or prompt injection; it is the model simply recalling memorized training data in response to a contextually leading prompt.

📄 Live demonstration link: chatgpt.com/share/1f044bb2-e7a1-4a17-898e-6f3cf659af0d

This example illustrates that training data extraction is not merely a theoretical concern — it can be demonstrated in practice against production systems. The extracted content may include copyrighted material, private communications, or personally identifiable information that should never appear in model outputs.



Summary: Key Takeaways from This Lecture

1

Adversarial Examples

Small, imperceptible perturbations to ML inputs can cause confident misclassification. No strong defense exists. Transferability means black-box models are not protected by opacity.

2

Prompt Injection

Adversarial inputs can confuse LLMs into taking harmful actions. Malicious prompts can be hidden in data, images, or base64-encoded text. Defense: sandbox model API calls using computer security techniques (CFI, dual LLMs).

3

Verifiable Training & Inference

Needed when models are trained or run by third parties. Options include hardware enclaves and fraud proof auditing. GPU non-determinism is a key technical challenge requiring recorded rounding directions.

4

Code Generation

LLM-generated code must go through security audit. AI assistants produce less secure code *and* create false confidence. CriticGPT-style approaches offer partial mitigation.

5

Model & Data Privacy

Model extraction via APIs is feasible and cheap. Models memorize training data that can be extracted by targeted queries — a serious compliance and privacy risk.

CHAPTER 7

Content Provenance

In an era of AI-generated imagery and deepfakes, how can we establish the authenticity and origin of digital media? This chapter examines the C2PA standard and cryptographic approaches to content provenance.



These look like prizewinning photos. They're AI fakes.

Artificially generated images of real-world news events proliferate on stock image sites, blurring truth and fiction

By [Will Oremus](#) and [Pranshu Verma](#)
November 23, 2023 at 6:00 a.m. EST

The Deepfake Problem

Generative AI has made it trivially easy to produce photorealistic images, videos, and audio that never existed. The examples above illustrate how AI-generated media can be virtually indistinguishable from authentic content — posing serious risks to journalism, elections, legal proceedings, and public trust.

Disinformation

Fabricated images of political figures, news events, or military actions can spread rapidly, shaping public opinion based on entirely false premises.

Legal Evidence

If digital media cannot be authenticated, its admissibility as evidence in court is fundamentally undermined. Anyone can claim any image is AI-generated.

Journalism Integrity

News organizations need to verify that images they publish are authentic photographs — not AI-generated fabrications passed off as real.

STANDARD

C2PA: A Standard for Content Provenance

How C2PA Works

The **Coalition for Content Provenance and Authenticity (C2PA)** is an open standard adopted by major technology and media companies. Supported by Adobe, Microsoft, the BBC, and camera manufacturers including Leica, Sony, and Nikon.

A C2PA-enabled camera embeds a **certified signing key** in its hardware. When a photo is taken, the camera cryptographically signs the image along with metadata (location, timestamp, camera model). The signature is embedded in the image file.

A viewer's device can verify the signature to confirm the image's authenticity and provenance — without trusting any intermediary.

The Chain of Trust

01

Camera Captures

Hardware signing key produces a certified signature over image + metadata.

02

Metadata Embedded

Signature, location, and timestamp are embedded directly in the image file.

03

Viewer Verifies

Any device checks the signature against the public key — confirming authenticity. Cloudflare added C2PA verification support in 2025.

BROADER ADOPTION

Not Just News Organizations: Why OpenAI Needs C2PA

Content provenance is not only a concern for photojournalists. AI image generators — including OpenAI's DALL-E and Sora — produce synthetic media that could be mistaken for authentic photographs. C2PA provides a mechanism to **mark AI-generated content as synthetic** at the point of creation.

The Use Case for AI Generators

OpenAI and similar companies can use C2PA to attach a signed provenance record to every AI-generated image — certifying that the image was produced by an AI system rather than a camera, and identifying the model version used.

Why This Matters

Without such marking, AI-generated images can be presented as authentic photographs. With C2PA, a viewer can verify: *"this image was generated by DALL-E 3, not a camera."* This creates a verifiable, unforgeable chain of custody for digital content.

LIMITATION

A Problem: Post-Processing Breaks Signatures

The Editing Problem

News organizations routinely process photos before publishing — resizing (e.g., 1500×1000), cropping, converting to grayscale, or blurring faces for privacy (AP lists a set of allowed operations). Each of these operations modifies the pixel data.

Because C2PA signatures are computed over the original pixel data, any post-processing **invalidates the signature**. A viewer's device cannot verify the signature on the processed photo — even though the editing was entirely legitimate.

C2PA's Current Solution and Its Limits

C2PA's current approach requires the editing software to re-sign the image after processing, attesting to what operations were performed. But this places trust in the editing software itself — a weaker guarantee than the original hardware-rooted signature.

The C2PA solution is not ideal. Is there a stronger cryptographic approach?

A Cryptographic Solution: zkSNARKs for Content Provenance

Zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs) provide a powerful solution: the editing software generates a cryptographic proof that the edited image was derived from an authentically signed original via permitted operations — without revealing the original image.

→ Public Statement

The laptop holds (**Edited**, **Ops**). The editing software attaches a proof π .

→ The Proof Claims

There exists a witness (**Orig**, **Sig**) such that: (1) Sig is a valid C2PA signature on Orig; (2) Edited is the result of applying Ops to Orig; (3) $\text{metadata(Edited)} = \text{metadata(Orig)}$.

→ Verification

The viewer's device verifies π and displays the authenticated metadata — without ever seeing the original unedited image. The proof is succinct and fast to verify.



PERFORMANCE RESULTS

Proofs for Post-Processing Operations: Practical Performance

Our work demonstrates that zkSNARK-based content provenance is practical for real-world high-resolution photography:

4 min

Proof Generation

Maximum proof generation time for a 6000×4000 image across resize, crop, grayscale, and blur operations using Adobe Photoshop algorithms.

~100KB

Proof Size

The resulting zkSNARK proof is compact — approximately 100 kilobytes — making it practical to embed in or attach to image files.

0.7s

Verification Time

Verification on a standard laptop takes just 0.7 seconds — fast enough for real-time browser-level content authentication.

Related work: PhotoProof (Kang et al., 2022), Monica et al. 2024. This remains an active research area with significant open problems in proof generation efficiency for video and real-time streaming content.

End of Lecture 8

This lecture covered the full attack surface of deployed AI systems — from training-time poisoning and adversarial examples, through prompt injection and agentic AI vulnerabilities, to model extraction, training data privacy, and content provenance.

Next Lecture

Consult the course schedule for upcoming topics. Continue reading the Stanford CS155 materials linked in the resources slide.

Key References

Goodfellow et al. '14 · Koh & Liang '17 ·
Tramèr et al. '19 · Carlini et al. '23 · C2PA
Standard · eprint.iacr.org/2024/1066

Critical Question

Whenever you encounter an ML system in production — ask: **what does it do about adversarial examples?** The answer tells you a great deal about its true security posture.